SWE OA Survival Guide: Weak Spots + Mnemonics

This guide focuses on the areas that still need reinforcement before the assessment:

- Debugging under pressure
- Set vs Hashmap distinction
- Sliding window invalid conditions
- DFS recursion flow
- Binary search movement logic
- Heap recognition
- Starter IDE skeleton reflexes

SET

- Trigger Words: duplicate, unique, already seen
- Mental Model: nightclub bouncer checking guest list
- Starter Skeleton: seen = set()
- Safe Lookups: python set add, python set remove
- Mnemonic: 'The bouncer recognizes faces, but does NOT count people.'

HASHMAP / DICTIONARY

- Trigger Words: frequency, most common, count occurrences
- Mental Model: scoreboard tracking points for EACH player
- Starter Skeleton: counts = {}
- Universal Skeleton: counts[item] = counts.get(item, 0) + 1
- Safe Lookups: python dict get method
- Mnemonic: 'The scoreboard is NOT the champion.'

SLIDING WINDOW

- Trigger Words: longest substring, contiguous, at most K
- Mental Model: moving car window
- Starter Skeleton: left = 0
- Core Thought: what makes the window INVALID?
- Safe Lookups: python while loop syntax, python set remove
- Mnemonic: 'One shove does not empty the car. Use WHILE not IF.'

TWO POINTERS

- Trigger Words: sorted array, target sum, pair sum
- Mental Model: snipers on opposite rooftops
- Starter Skeleton: left = 0, right = len(nums) - 1
- Movement Logic: too small -> left++, too big -> right--
- Mnemonic: 'Cold goes bigger, hot goes smaller.'

DFS / ISLANDS

- Trigger Words: islands, connected, grid, flood fill
- Mental Model: zombie infection spreading
- Starter Skeleton: def dfs(r, c):
- DFS Goal: eat the whole island once
- Safe Lookups: python recursion syntax
- Mnemonic: 'Zombies keep rebiting each other forever unless visited.'

BINARY SEARCH ON ANSWER

- Trigger Words: minimum possible, smallest speed, within h hours
- Mental Model: The Price Is Right guessing game
- Starter Skeleton: left = 1, right = max(piles)
- Core Thought: does mid WORK?
- Movement Logic: works(mid) -> move right, fails(mid) -> move left
- Mnemonic: 'If it works, try cheaper.'

HEAP

- Trigger Words: top K, priority, most frequent
- Mental Model: VIP line
- Starter Skeleton: import heapq
- Safe Lookups: python heapq syntax
- Mnemonic: 'Top K means VIP access.'

STACK

- Trigger Words: parentheses, nested, matching brackets
- Mental Model: plate stack
- Starter Skeleton: stack = []
- Safe Lookups: python list pop
- Mnemonic: 'Last opened, first closed.'

DEBUGGING WEAK SPOTS

- Biggest Bug: singular vs plural variables
- Example: for num in nums -> use num inside loop
- Wrong Return Bug: return count, not nums
- Common Syntax Bugs: missing colons, capitalization, indentation

- Mnemonic: 'Talk to the current player, not the whole team.'

Final Assessment Strategy

1. Recognize the pattern FAST.
2. Start typing the skeleton immediately.
3. Use docs only for syntax help.
4. Working code beats perfect code.
5. Do not spiral during debugging.